

```

"""
cross_reference_engine.py
=====
GRUS Audit Pipeline – Cross-Reference Engine
Specification: GRUS-CRX-001-v1.0

Auto-detects every prior repository element a submission depends on,
generates the citation manifest, propagates credit through the dependency
tree, and produces the auto-citation artifacts. Replaces author-discretionary
citation with mechanically-determined dependency tracing.

Published by: Green Recursive Utility Service LLC
"""

import os
import ast
import json
from typing import Dict, List, Set, Any
from dataclasses import dataclass, field, asdict

@dataclass
class CitationManifest:
    """The auto-generated citation manifest for a passing submission."""
    domain_id: str
    domain_title: str
    contributor: Dict[str, Any] = field(default_factory=dict)
    constants_used: List[Dict] = field(default_factory=list)
    datasets_cited: List[Dict] = field(default_factory=list)
    prior_domains_touched: List[Dict] = field(default_factory=list)
    confirmations_leveraged: List[Dict] = field(default_factory=list)
    new_pending_predictions: List[Dict] = field(default_factory=list)
    cross_domain_consistency: Dict = field(default_factory=dict)

def detect_constant_dependencies(repo_path: str) -> Set[str]:
    """
    Static AST analysis to extract every constant imported from the
    submission's constants module (svcf_constants.py).
    """
    used_constants: Set[str] = set()
    for root, _, files in os.walk(repo_path):
        for fname in files:
            if not fname.endswith(".py"):
                continue
            full_path = os.path.join(root, fname)
            try:
                with open(full_path, "r", encoding="utf-8") as f:

```

```

        source = f.read()
        tree = ast.parse(source)
    except (SyntaxError, UnicodeDecodeError):
        continue
    for node in ast.walk(tree):
        if isinstance(node, ast.ImportFrom):
            if node.module and "svcf_constants" in node.module:
                for alias in node.names:
                    if alias.name == "*":
                        continue
                    used_constants.add(alias.name)
    return used_constants

def detect_dataset_dependencies(manifest: Dict) -> List[Dict]:
    """Read manifest cited datasets and emit normalized records."""
    out: List[Dict] = []
    for d in manifest.get("datasets_cited", []):
        url = d.get("url", "")
        out.append({
            "url": url,
            "description": d.get("description", ""),
            "access_statute": d.get("access_statute", "OSTP 2022 Public Access Mandate"),
            "first_cited_by": "self", # ledger lookup updates this
        })
    return out

def query_ledger_for_constant_users(constant: str, ledger_snapshot: Dict) -> List[Dict]:
    """
    Find every prior ledger entry that uses the same constant.

    ledger_snapshot is a dict keyed by domain_id, with values containing
    the prior submission's manifest. In production this is loaded from
    the actual ledger files.
    """
    users = []
    for prior_id, prior_record in ledger_snapshot.items():
        prior_constants = prior_record.get("constants_used", [])
        for c in prior_constants:
            if c.get("symbol") == constant:
                users.append({
                    "domain": prior_id,
                    "shared_element": constant,
                    "reason": f"shares constant {constant}",
                    "contributor": prior_record.get("contributor", {}).get("name", "unknown"),
                })
        break
    return users

```

```
def query_ledger_for_dataset_users(dataset_url: str, ledger_snapshot: Dict) -> List[Dict]:
    """Find every prior ledger entry that cites the same dataset."""
    users = []
    for prior_id, prior_record in ledger_snapshot.items():
        prior_datasets = prior_record.get("datasets_cited", [])
        for d in prior_datasets:
            if d.get("url") == dataset_url:
                users.append({
                    "domain": prior_id,
                    "shared_element": dataset_url,
                    "reason": "shares dataset",
                    "contributor": prior_record.get("contributor", {}).get("name", "unknown"),
                })
            break
    return users
```

```
def query_ledger_for_confirmations(manifest: Dict, ledger_snapshot: Dict) -> List[Dict]:
    """
    Find which C-confirmations the submission's mechanism leverages.

    The manifest's predictions list may reference C-IDs explicitly;
    additionally, any constant traced back to a C-confirmation source
    is implicitly leveraging that confirmation.
    """
    confirmations = []
    explicit = manifest.get("confirmations_leveraged", [])
    for c_id in explicit:
        record = ledger_snapshot.get(c_id, {})
        contributor = record.get("contributor", {}).get("name", "original-anchor-contributor")
        confirmations.append({
            "confirmation": c_id,
            "original_contributor": contributor,
            "credit": f"explicit dependency on {c_id}",
        })
    return confirmations
```

```
def generate_citation_manifest(
    repo_path: str,
    manifest: Dict,
    ledger_snapshot: Dict,
    domain_id: str,
    contributor_doi: str,
    audit_id: str,
    audit_date: str,
) -> CitationManifest:
    """
    Generate the full auto-citation manifest for a passing submission.
```

```

"""
# 1. Constant dependencies
used_constants_set = detect_constant_dependencies(repo_path)
constants_used = []
for c_name in sorted(used_constants_set):
    constants_used.append({
        "symbol": c_name,
        "from": "svcf_constants.py",
        "classification": "EXACT" if c_name in {"B", "BETA", "PSI", "K_TD", "K_MODE"} else
"MEASURED",
        "original_priority": "rxiv:2602.0018 (2025-11-16)",
    })

# 2. Dataset dependencies
datasets_cited = detect_dataset_dependencies(manifest)
for d in datasets_cited:
    prior_users = query_ledger_for_dataset_users(d["url"], ledger_snapshot)
    if prior_users:
        d["first_cited_by"] = prior_users[0]["domain"]

# 3. Domain dependencies (via shared constants)
prior_domains_touched: List[Dict] = []
seen_domains: Set[str] = set()
for c in constants_used:
    users = query_ledger_for_constant_users(c["symbol"], ledger_snapshot)
    for u in users:
        if u["domain"] not in seen_domains:
            seen_domains.add(u["domain"])
            prior_domains_touched.append(u)

# 4. Confirmation dependencies
confirmations_leveraged = query_ledger_for_confirmations(manifest, ledger_snapshot)

# 5. New pending predictions
new_pending = [
    p for p in manifest.get("predictions", [])
    if p.get("status") == "pending"
]

# 6. Cross-domain consistency record (placeholder; populated by closure_tester)
cross_domain_consistency = {
    "re_verification_triggered": [d["domain"] for d in prior_domains_touched],
    "re_verification_result": "PENDING", # set by closure_tester
    "cumulative_p_before": "",
    "cumulative_p_after": "",
}

return CitationManifest(
    domain_id=domain_id,
    domain_title=manifest.get("submission_title", ""),

```

```

contributor={
    "name": manifest.get("contributor_name", ""),
    "type": manifest.get("contributor_type", "individual"),
    "affiliation": manifest.get("affiliation", ""),
    "doi": contributor_doi,
    "doi_timestamp": manifest.get("doi_timestamp", ""),
    "audit_id": audit_id,
    "audit_date": audit_date,
},
constants_used=constants_used,
datasets_cited=datasets_cited,
prior_domains_touched=prior_domains_touched,
confirmations_leveraged=confirmations_leveraged,
new_pending_predictions=new_pending,
cross_domain_consistency=cross_domain_consistency,
)

```

```

def write_citation_manifest(manifest: CitationManifest, output_path: str) -> str:

```

```

    """
    Write the citation manifest as YAML to output_path.
    Returns the path written.
    """
    import yaml
    data = asdict(manifest)
    with open(output_path, "w", encoding="utf-8") as f:
        yaml.safe_dump(data, f, sort_keys=False, default_flow_style=False)
    return output_path

```

```

if __name__ == "__main__":
    import sys
    import yaml as _yaml
    if len(sys.argv) < 4:
        print("Usage: python cross_reference_engine.py <repo_path> <ledger.json> <output.yaml>")
        sys.exit(0)
    repo = sys.argv[1]
    with open(os.path.join(repo, "GRUS_MANIFEST.yaml")) as f:
        manifest = _yaml.safe_load(f)
    with open(sys.argv[2]) as f:
        ledger = json.load(f)
    cm = generate_citation_manifest(
        repo_path=repo,
        manifest=manifest,
        ledger_snapshot=ledger,
        domain_id="DXX",
        contributor_doi=manifest.get("doi", ""),
        audit_id="GRUS-AUDIT-NNNN",
        audit_date="2026-MM-DD",
    )

```

```
out = write_citation_manifest(cm, sys.argv[3])  
print(f"Citation manifest written to {out}")
```